FR. CONCEICAO RODRIGUES COLLEGE OF ENGINEERING
Department of Computer Engineering | Academic Year 2026

TESSERA: Data Structures Mini-Project Presentation Script

Subject: Data Structures (SE Computer Engineering C) | **Guide:** Prof. Ashwini Pansare
Project: Championship Scrabble Engine & Position-Keyed Belief Lattice in ISO C11 (2016 Elo)

1. Team Member Allocation & Module Ownership

Member	Assigned Module & Data Structures	Key C Source Files	Presentation Role
Harshit Khandelwal (Team Lead)	Minimal DAWG Lexicon (Trie) & Appel-Jacobson Movegen with 32-bit Bitmasks	`src/dict.c` `src/movegen.c`	Intro, Problem Statement & Lexicon / Movegen Architecture
Prem	Dual-Accumulator Evaluator, 8-Phase Arrays & Deep Dual-Headed NNUE SIMD	`src/eval.c` `src/nnue.c`	Static Evaluation, Bigram Synergy Matrix & Neural Inference
Ashley	Pure Position-Keyed Belief Lattice (DAG) & Priority Queue (Binary Max-Heap)	`src/lattice.c` `src/settle.c`	DAG Transposition Merging & DirtyQueue Bellman Relaxation
Aaryan	Exact Transposition DAG Endgame Solver, Bayesian Inference & Benchmark	`src/endgame.c` `src/court.c`	Negamax Endgame Solver, Tournament Matrix & 2016 Elo Rating

2. Course Curriculum Alignment Matrix

Data Structure	Engine Implementation	Time Complexity	Space Complexity
Trie / Automaton	Minimal SOWPODS DAWG (185,702 edges bit-packed into 4-byte integers)	O(L) string search	742 KB flat array
Hash Table	64-bit Zobrist Hash Table with open addressing and linear probing	O(1) average lookup	16,384 buckets (load < 0.08)
Directed Graph (DAG)	Position-Keyed Belief Lattice with Gaussian beliefs (μ, σ^2, N) and back-pointers	O(V + E) traversal	Merged graph nodes
Binary Max-Heap	Prioritized DirtyQueue for Bellman backward induction relaxation waves	O(log N) push/pop	2,048 fixed heap array
Bitwise Sets / Masks	32-bit cross-check masks for instantaneous orthogonal word validation	O(1) bitwise AND	900 bytes (15x15 board)
Transposition Cache	32,768-entry depth-preferred cache for exact terminal endgame negamax	O(1) query / store	786 KB (32k x 24B)
Static Memory Pool	Depth-indexed static pool (g_endgame_moves_pool) eliminating stack frames	O(1) frame indexing	80 bytes vs 1MB stack

3. Word-for-Word Presentation Speaking Scripts

Each team member has a dedicated 2-minute speaking segment with exact cues and key technical terms.

SPEAKER 1: Harshit Khandelwal (Team Lead) — 2.5 Minutes

Topic: Executive Introduction, Problem Statement & Minimal DAWG / Movegen Architecture

*"Respected Prof. Ashwini Ma'am and fellow classmates, good morning. Today, our team is presenting our Data Structures Mini Project: **Tessera — a Grandmaster-tier Scrabble Intelligence Engine implemented entirely in standard ISO C11 with zero external libraries**.*

*When people think of board game AI like Chess or Go, they think of Minimax trees or Monte Carlo Search Trees. However, Scrabble presents a fundamentally harder data structure challenge: it is an imperfect-information game, the branching factor is massive with over 2,500 legal moves per turn, and independent row placements cause move-order transpositions to explode exponentially. To solve this, our project makes a radical architectural choice: **we strictly eliminated search trees**. Instead, we built a 100% position-keyed Directed Acyclic Graph (Belief Lattice) powered by a Binary Max-Heap, a Minimal DAWG, and Zobrist Hash Tables.*

*I will now explain our first primary data structure: **The Minimal Directed Acyclic Word Graph (DAWG)** in src/dict.c. In Scrabble, verifying words using a binary search tree or array takes O(log N) per word. A standard Trie for the 267,751-word tournament lexicon would require over 40 Megabytes of pointers! Our DAWG merges both common prefixes and common suffixes. Suffixes like -ING or -ATION are shared across thousands of words and collapsed into identical graph nodes. The entire 267,751-word lexicon is compressed into only **185,702 edges fitting into 742 Kilobytes**! Each edge is bit-packed into a single 32-bit integer: 24 bits for the child index, 5 bits for the letter, 1 bit for word termination, and 1 bit for end-of-node. This flat array loads in 0.0001 seconds via zero-copy binary ingestion. Combined with our Appel-Jacobson move generator in src/movegen.c, which precomputes 32-bit bitmasks for vertical cross-checks on the 15x15 board, Tessera generates over **1.3 Million legal moves per second** in pure C!*

*I now pass the presentation to **Prem** to explain our static evaluation data structures."*

SPEAKER 2: Prem — 2.0 Minutes

Topic: Dual-Accumulator Evaluator, 8-Phase Arrays, Bigram Synergy Matrix & NNUE

"Thank you, Harshit. A common flaw in game engines is treating piece values statically. In Scrabble, tile values change drastically as the tile bag empties. A blank tile is worth +26 points in the opening because it triggers multiple 50-point bingos, but near bag exhaustion, its value drops to +14 points because there is no time left to compound it.

*In src/eval.c, we designed a **Dual-Accumulator Evaluator** parameterized across **8 discrete game-phase buckets**:*
*• **The Board Accumulator** evaluates the 15x15 grid for immediate word score, premium square occupancy (Triple Word, Double Word), and hazard exposure (such as floating vowels adjacent to unblocked Triple Words).*
*• **The Rack Leave Accumulator** evaluates unplayed tiles using an 8x27 lookup array of phase-dependent tile values, a 27x27 **Pairwise Synergy Matrix** capturing multi-tile harmonies (like QU synergy +10 pts or S+Blank +4.5 pts), and non-linear duplicate penalties (VV -36.0 pts, WW -20.0 pts).*
*• **Sub-Rack Exchange Optimizer:** When a rack is choked, we evaluate all 2^k subsets ($k \leq 7$, up to 128 subsets) via bit manipulation to find the mathematically optimal tile discard split.*

Furthermore, in src/nnue.c, we implemented an Efficiently Updatable Neural Network (NNUE) with a $448 \rightarrow 128 \rightarrow 32 \rightarrow 2$ dual-headed architecture using AVX2 SIMD vector instructions in pure C, ensembled 50/50 with our hand-tuned heuristics.

*I now hand over to **Ashley** to explain our project's core innovation: The Belief Lattice DAG."*

SPEAKER 3: Ashley — 2.5 Minutes

Topic: Pure Position-Keyed Belief Lattice (DAG), Zobrist Hashing & Binary Max-Heap

*"Thank you, Prem. This brings us to the core data structure novelty of Tessera: **The Pure Position-Keyed Belief Lattice**.*

In traditional game trees, if move sequence A then B leads to the exact same board state as B then A, a search tree allocates

two distinct nodes and evaluates both independently. In Scrabble, where words are placed across independent rows, this causes an exponential explosion of redundant branches!

In src/lattice.c, we strictly prohibited game trees. Instead:
*1. Every 15x15 board configuration is hashed into a 64-bit integer using **Zobrist Hashing** with pseudo-random keys.*
*2. States are inserted into an **Open-Addressing Hash Table** with 16,384 slots. If a board has already been reached through another move order, **the nodes merge by construction**. Transposition merging is 100%!*
3. Each node in the DAG stores a continuous Gaussian belief: $B(v) = (\mu, \sigma^2, N)$.

How do updates propagate backward in a DAG?
*In a tree, each node has only one parent. In a DAG, a node has multiple parents! In src/settle.c, we implemented a **Binary Max-Heap (DirtyQueue)**. Whenever simulation updates a node's belief, the change magnitude $|\Delta|$ is pushed into the Max-Heap in O(log N) time. We extract the maximum-priority dirty node and relax its Q-value using our calibrated Bellman operator: $Q(e) = \text{MoveStatic}(e) + \gamma \times \Delta_\text{combined}(e)$. All parent back-pointers are enqueued into the heap. Tactical discoveries ripple backward across the graph in prioritized waves until reaching equilibrium!*

*Furthermore, we implemented **2-Ply Draw Realization**: during deliberation, Tessera simulates refilling its rack from the remaining bag tiles and tests its actual turn-2 response equity on the resulting board.*

*I now invite **Aaryan** to cover the Endgame Solver, Bayesian Inference, and our Tournament Results."*

SPEAKER 4: Aaryan — 2.0 Minutes

Topic: Exact Endgame DAG Solver, Bayesian Opponent Tracker & Tournament Results

*"Thank you, Ashley. In the final phase of Scrabble, when the bag is empty and the opponent has ≤ 7 tiles, Scrabble transitions into a **deterministic perfect-information game**.*

*In src/endgame.c, we built an **Exact Negamax Endgame Solver** running over a **32,768-entry Transposition Table**:*
• We eliminated stack overflow risks by creating a depth-indexed static memory pool (g_endgame_moves_pool), reducing memory usage from 1 MB per recursive stack frame to just 80 bytes!
*• The solver searches up to 8 plies deep and computes the exact optimal move and terminal margin in **under 1.5 milliseconds**!*
*• In src/court.c, our engine stops deliberating using **The Root Court** — a Bayesian hypothesis test drawing 1,000 Thompson samples. When $P(\text{best move}) \geq 98\%$, it adjourns instantly, saving compute for contested turns.*

Empirical Tournament Benchmark (70 Games vs 7 Bot Profiles):
We benchmarked Tessera across 70 tournament matches against specialized bots, including commercial simulation engines:
*• Against **MAGPIE / Woogles 2-Ply Realization (~2150 Elo)**, Tessera won **6 - 4 (+23.3 pts spread)**.*
*• Against **Defensive Turtler**, Tessera scored a perfect **10 - 0 sweep (+74.9 pts spread)**.*
*• Against **Neural Static 1-Ply**, Tessera won **8 - 2 (+60.4 pts spread)**.*
*• **Overall Record: 50 Wins, 20 Losses (71.4% Win Rate)** averaging **460.1 points per game** and **2.14 bingos per game**.*
*• **Official NASPA Performance Rating (TPR):** Evaluated against the 1857-rated field: **~2016 Elo** (National Champion Tier)!*

We will now begin our live demonstration. Thank you!"

4. Live Demonstration Execution Steps

Step & Command	Demonstration Target	Presenter & Speaking Cue
Step 1: Test Suite `make test`	Executes all 5 C unit tests: DAWG (0.0001s), Movegen (2,916 plays in 0.84ms), Eval, Lattice proof, and Endgame solver (1.57ms).	**Harshit:** *"Ma'am, as you can see, all 5 core data structures compile cleanly with zero external libraries and pass in under 2 seconds."*
Step 2: Live Match `./tessera ai-vs-ai`	Runs a live autonomous game showing DAG telemetry: nodes created, transpositions merged, dirty passes settled, and Court confidence.	**Ashley:** *"Notice how identical board states reached through different word placements merge into single vertices without duplicate tree branches."*
Step 3: Visual Guide `xdg-open readme_preview.html`	Displays standalone rendered documentation with GitHub CSS, KaTeX mathematical equations, and Mermaid graph architecture diagrams.	**Aaryan & Prem:** *"Here is our full 70-game tournament matrix showing our 71.4% win rate and verified 2016 Elo rating."*

5. Personalized Teacher Viva Defense Cards

Anticipated tough questions from Prof. Ashwini Pansare, mapped directly to each team member:

Harshit Khandelwal — Lexicon, DAWG & Movegen Viva Card

Q: Why did you choose a DAWG over a standard Trie or Binary Search Tree?

A: A BST requires O(log N) string comparisons where each comparison is O(L), making generation across millions of permutations too slow. A standard Trie shares prefixes, but for SOWPODS (267,751 words) it requires over 400,000 nodes and tens of megabytes of pointer overhead. A DAWG is a minimal deterministic finite state automaton that merges both common prefixes AND common suffixes. Suffixes like -ING, -ED, or -ATION share the exact same child subgraphs, compressing the entire dictionary into just 185,702 4-byte packed edges (742 KB) while preserving O(L) verification directly inside CPU L2 cache.

Prem — Evaluator, Feature Matrices & SIMD Viva Card

Q: Why decouple evaluation into 8 game phases instead of a single static point table?

A: Tile equity is non-stationary. In the Opening (80-100 tiles), a blank is worth +26.5 points because it enables multiple future bingos. In Exhaustion (1-9 tiles), it drops to +14 points because there is no time left to compound its value. Conversely, heavy tiles like Q and V are manageable early on, but catastrophic near bag depletion. The 8-phase bucket model adapts tile synergies, vowel-consonant ratios, and stem bonuses dynamically to the exact state of the bag.

Ashley — Belief Lattice DAG & Priority Queue Heap Viva Card

Q: How does the Binary Max-Heap operate in your DAG and what are its complexities?

A: Our Max-Heap is implemented in src/settle.c as DirtyQueue. Unlike a tree where updates only have one parent, a DAG node can have multiple parents. Whenever a node's belief moves by |Δ|, its parent indices are pushed into the heap with priority |Δ| in O(log N) time using bubble-up (heap_swap). We extract the maximum element in O(log N) time using bubble-down. This guarantees that the most significant tactical refutations are relaxed backward first, achieving numerical Bellman equilibrium in minimal passes.

Aaryan — Endgame Negamax, Memory Pool & Elo Viva Card

Q: How did you prevent stack overflow during deep recursive endgame search?

A: In src/endgame.c, generating legal moves requires a MoveList struct holding up to 8,192 moves (~1 Megabyte). Initially, allocating MoveList moves as a local stack variable inside recursive negamax overflowed the Linux 8 MB stack at depth 5. We resolved this by creating a depth-indexed static pool (static MoveList g_endgame_moves_pool[MAX_ENDGAME_DEPTH + 2]). Each recursive frame at depth d indexes directly into slot d of the pre-allocated static pool, slashing stack consumption

from 1 Megabyte to just 80 bytes per frame and solving 8-ply endgames in under 1.5 milliseconds.